



ITESO

Universidad Jesuita
de Guadalajara

Networks for embedded systems

Thread practice

Students:

- Roberto Agustín Franco Ibáñez #741059
agustin.franco@iteso.mx
- Eduardo Guzman Villalobos # 750750
eduardo.guzmanv@iteso.mx
- Santiago Salcedo Torres #728786
santiago.salcedo@iteso.mx

Teacher:

- Estephania Martínez García

Due date: 07/10/2026

Content

Thread practice	1
Statement	4
PART 1	4
PART 2	4
PART 3	5
Thread concepts	7
Thread	7
Mesh Network	7
6LoWPAN	7
UDP	7
Leader	7
Active Router	7
Router Eligible End Device (REED)	8
End Device	8
Commissioner	8
Joiner	8
IPv6 Addresses (Link-Local, Mesh-Local, Multicast)	8
CoAP (Constrained Application Protocol)	8
URI (Uniform Resource Identifier)	8
CoAP Callback	9
CON (Confirmable) and NON (Non-confirmable) Messages	9
Detailed network	9
Devices & types	9
IPv6 addresses	9
PanID	9
Channel	9
Network name	9
Modified network	10
Printed request in shell	10

Timer on Router2 to request counter	10
Printed response in the shell on Router 2	10
Network diagram	11
Sequence diagram	11
Terminal screenshots	12
Sniffer screenshots (joining process).....	13
Conclusions.....	15
References.....	15

Statement

PART 1

Leader/Router1:

1. Create your own CoAP URI with your team's name. ie `"/team1"`
2. Start a Timer that will have a counter from 1 to 150. This counter will be accessible to the network through your URI `"/team1"`. (1 second)
3. When a CoAP instruction is received, print in the shell the IP address of the requester and indicate if it was a CON or NON request. *CON instruction received from fd01:abcd::1234*
4. If it was a CON request, reply with a CoAP Ack, if it was a NON request, don't reply with a CoAP Ack.

Router2:

1. Once you are connected to the network, start an Interval Timer that will REQUEST the current value stored in your team's URI repo from the Router.(2 seconds)
2. Print in shell the value that was requested and the src IP address of the packet including the type of request made *Counter = 43 from fd01:abcd::1111 type CON/NON*

PART 2

Router2:

- Create another CoAP URI called `"/stopMyTmr"` to stop the timer for the request for the leader
- Create another CoAP URI called `"/stopTmr"` to stop the timer of the leader, print the information *"Timer stopped from fd01:abcd::1111 type CON/NON Count= 43 "*
- Create another CoAP URI called `"/startTmr"` to re-start the timer of the leader , the count should start with 0, print the information *" Timer started from fd01:abcd::1111 type CON/NON starting Count= 43"*
- Create another CoAP URI called `"/restartTmr"` to re-start the timer of the leader, the count should continue in the value that was left when stopped, print the information *" Timer started from fd01:abcd::1111 type CON/NON starting Count= 43"*

PART 3

Leader/Router1:

1. Initialize the accelerometer from the board on your application
2. Create another CoAP URI called "/accel"
3. Prepare the CoAP callback to reply with the x, y, z raw reading when a CON/NON GET request was received and print the type of request as well as the address of the requester

Router2:

1. Send a COAP CON/NON GET request to leader/router1 to URI "/accel" you should receive the accelerometer raw reading and print it with the following format Ie. X=100 Y=100 Z=0 from IPv6 address: fd17::abcd:111 CON/NON request
2. Turn on led according to the highest axis
 1. Y magenta led
 2. Z cyan led
 3. X green led

If values the same , turn LEDs accordingly

For the teams of 3 people this point is mandatory, for the ones of 2 people this is extra points

- Add an extra URI "/changeTime 3" for the PART II, to set the timer and change the interval in seconds.

(User you assigned team channel)

The report will include:

- Thread concepts used
- Detailed network topology of the final setup
 - Devices & types
 - IPv6 addresses
 - PanID
 - Channel
 - Network name
- Modified code on both devices (Leader/Router 1 and Router 2)

- Printed request in shell
 - Timer on Router2 to request counter
 - Printed response in the shell on Router 2
 - Network diagram
 - Sequence diagram
- Terminal screenshots, displaying the CoAP messages being sent/received
- Sniffer screenshots of the entire joining process.***extra
- Sniffer screenshots, displaying the CoAP messages being sent/received with a detailed description of the information you can obtain***extra
- Personal conclusions
 - Is Thread useful? Why?
 - What went wrong in the process?
 - Was it easy to implement?

*Remember to use a GitHub repository TODAY with the initial commit.

*All team members must do commits.

Thread concepts

Thread

Is a wireless mesh network protocol designed for smart home devices. It is built on top of IEEE 802.15.4 and uses IPv6, so every device has its own IP address. It is made to be low power consumption and to keep working even if one device fails so it is very robust.

Mesh Network

A mesh network is a network where devices connect to several neighbors instead of to a single central point. Messages can jump from device to device until they reach their destination. If one device fails, the network can find another path.

IEEE 802.15.4

This is the radio standard that Thread uses for its physical and MAC layers. It is designed for low power, short-range communication.

6LoWPAN

6LoWPAN is an adaptation layer that lets IPv6 work over 802.15.4. IPv6 packets are too big for the small 802.15.4 frames, so 6LoWPAN compresses the headers and splits large packets into fragments.

UDP

UDP is a transport protocol that sends data without first establishing a connection. It is fast and light, but it does not guarantee delivery.

Leader

The Leader is the router that manages the network. It assigns router IDs and keeps the network parameters. In our lab, Router1 created the network, so it became the Leader. If the Leader fails, another router takes its place, so there is no single point of failure.

Active Router

An active router is a device that forwards messages between other devices in the Thread network, and it keeps the mesh connected. Routers are usually powered all the time.

Router Eligible End Device (REED)

A REED is a device that can work as a router but starts as a regular node after joining. If the network needs more routers, it asks the Leader to be promoted.

End Device

An end device is a simple node that does not forward messages for others. It talks to the network only through its parent router. A Sleepy End Device is a special type that turns off its radio most of the time to save battery.

Commissioner

The Commissioner is the device that is allowed to authenticate new devices and give them the network credentials. It acts as the "security guard" of the network, since a new device can only join if the Commissioner approves it.

Joiner

A Joiner is the new device that wants to enter a Thread network. It starts without the network credentials and receives them after being authorized by the Commissioner.

IPv6 Addresses (Link-Local, Mesh-Local, Multicast)

Every Thread device gets several IPv6 addresses. The link-local address only works with direct neighbors (one hop). The mesh-local address (ML-EID) works anywhere inside the Thread network, and it is the one recommended for applications. Multicast addresses send a message to many devices at once. We can see all these addresses with the `ifconfig` command.

CoAP (Constrained Application Protocol)

CoAP is a lightweight protocol, like a simplified HTTP, made for small devices. It works with requests and responses using methods like GET, POST, PUT, and DELETE.

URI (Uniform Resource Identifier)

A URI is the "name" or path of a resource on a CoAP device, like `/team2` or `/accel`. When a device sends a request to a URI, the receiving device runs the function linked to it.

CoAP Callback

A callback is the function that runs automatically when a CoAP message arrives at a certain URI. Inside it, we check what type of request it was (GET, POST, etc.), do the needed action, and, if required, send a reply.

CON (Confirmable) and NON (Non-confirmable) Messages

These are the two main types of CoAP requests. A CON message asks the receiver to confirm that it arrived, so the receiver must answer with an ACK. A NON message does not need confirmation, so no ACK is sent.

Detailed network

Devices & types

There are 2 devices in total, both are routers (Router1 and Router2). When the network starts up, one will assume the role of leader and the other that of router.

IPv6 addresses

	Router1	Router2
LL64	fe80::e804:6b67:4d56:92f2	fe80::ed03:ab31:c53b:95c
ML64	fd3d:8db0:57a0:26d5:c0ba:2962:a5e0:6e5f	fd3d:8db0:57a0:26d5:2d5a:c43b:c3e2:433f
ML16	fd3d:8db0:57a0:26d5:2d5a:c43b:c3e2:433f	fd3d:8db0:57a0:26d5::ff:fe00:400

PanID

PAN ID: 0x2222

Channel

Channel: 15

Network name

Network name: team2

Modified network

Printed request in shell

This is where the code is modified to print a message indicating that an instruction was received, depending on whether it is CON or NON.

```
if(pSession->msgType == gCoapConfirmable_c)
{
    shell_printf("CON instruction received from %s\r\n", addrStr);
}
else if(pSession->msgType == gCoapNonConfirmable_c)
{
    shell_printf("NON instruction received from %s\r\n", addrStr);
}
```

Timer on Router2 to request counter

And here is the function used to request de counter on router 2

```
/* Function used by Timer Task to request the Counter */
void APP_TriggerCounterRequest(void)
{
    if(mLeaderAddressValid == TRUE)
    {
        (void)NWKU_SendMsg(
            APP_RequestTeamCounter,
            NULL,
            mpAppThreadMsgQueue
        );
    }
}
```

Printed response in the shell on Router 2

```
shell_printf(
    "Counter = %s from %s type CON\r\n",
    counterStr,
    addrStr
);
```

Network diagram

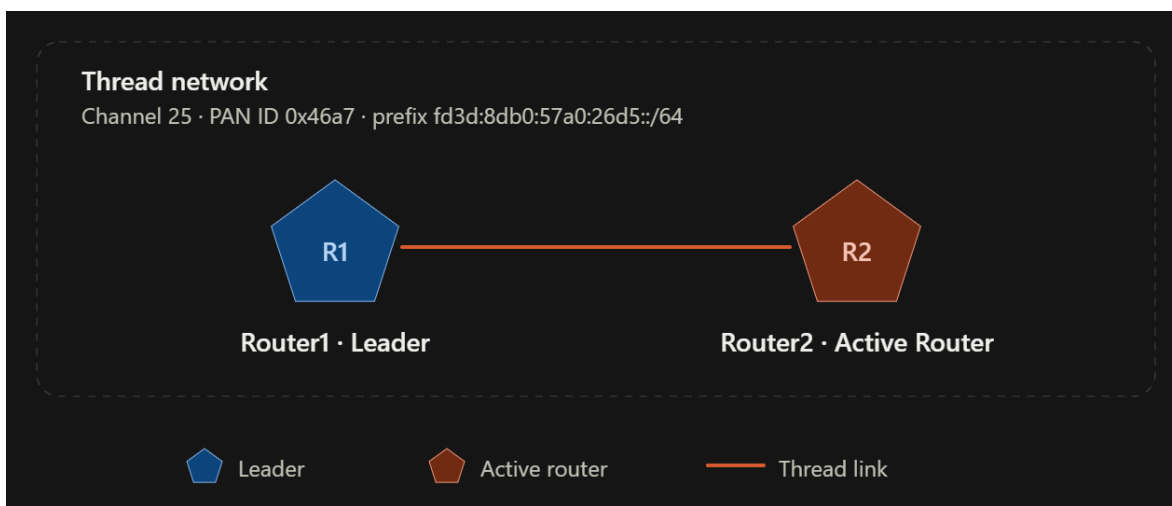


Figure 0: Network diagram (IA generated[3]).

Sequence diagram

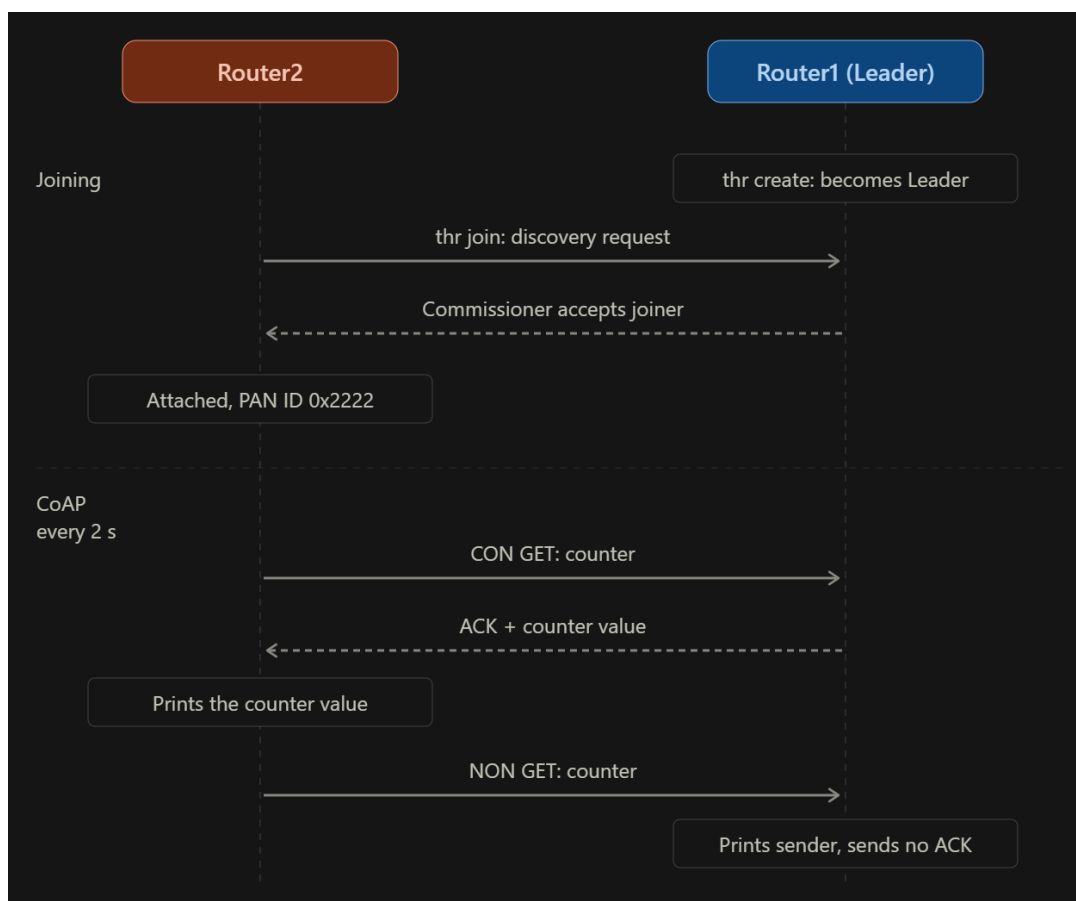


Figure 1: Sequence diagram (IA generated[3]).

Terminal screenshots

```
SHELL starting...
NXP Thread v1.1.2.36
Enter "thr join" to join a network, or "thr create" to start new network
Enter "help" for other commands

$ thr set channel 15
Success!
$ thr create
Creating network...
$
Node has taken the Leader role
Created a new Thread network on channel 15 and PAN ID:0x2222

Interface 1: 6LoWPAN
    Mesh local address (ML64): fd35:326e:a7ea:de92:8dc2:8707:af8f:9565
    Mesh local address (ML16): fd35:326e:a7ea:de92::ff:fe00:0
Interface 0: Loopback
(Local) Commissioner Started

Joiner accepted

$ CON instruction received from fd35:326e:a7ea:de92::ff:fe00:1
$ NON instruction received from fd35:326e:a7ea:de92::ff:fe00:1
$ CON instruction received from fd35:326e:a7ea:de92::ff:fe00:1
$ □
```

Figure 2: Router1 creating network.

```
SHELL starting...
NXP Thread v1.1.2.36
Enter "thr join" to join a network, or "thr create" to start new network
Enter "help" for other commands

$ thr set channel 15
Success!
$ thr join
Joining network...
$
Commissioning successful

Attached to network with PAN ID: 0x2222
Leader ALOC configured: fd35:326e:a7ea:de92::ff:fe00:fc00

$ CON

$ Counter = 33 from fd35:326e:a7ea:de92::ff:fe00:fc00 type CON

$ NON

$ CON

$ Counter = 37 from fd35:326e:a7ea:de92::ff:fe00:fc00 type CON

$ NON

$ CON
```

Figure 3: Router2 joining network.

Sniffer screenshots (joining process)

1	0.00000000	fe80::14e2:ff5:d6a3...	ff02::1	MLE	83	
2	0.000017400	fe80::14e2:ff5:d6a3...	ff02::1	MLE	83	
3	4.441788300	fe80::14e2:ff5:d6a3...	ff02::1	MLE	83	
4	4.441802200	fe80::14e2:ff5:d6a3...	ff02::1	MLE	83	
5	7.563065300	fe80::ca8:d571:199:...	ff02::1	MLE	51	Discovery Request
6	7.563088000	fe80::ca8:d571:199:...	ff02::1	MLE	51	Discovery Request
7	7.685237100	fe80::14e2:ff5:d6a3...	fe80::ca8:d571:199:...	MLE	97	Discovery Response
8	7.685257400	fe80::14e2:ff5:d6a3...	fe80::ca8:d571:199:...	MLE	97	Discovery Response
9	7.685311800			IEEE 8...	19	Ack
10	7.685321100			IEEE 8...	19	Ack
11	9.811233400	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	139	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
12	9.811254700	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	139	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
13	9.811302800			IEEE 8...	19	Ack
14	9.811314500			IEEE 8...	19	Ack
15	9.826490700			IEEE 8...	19	Ack
16	9.826505900			IEEE 8...	19	Ack
17	9.826929700	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	138	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
18	9.826936000	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	138	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
19	9.836332800	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	138	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
20	9.836357100	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	138	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
21	9.836413700			IEEE 8...	19	Ack
22	9.836423900			IEEE 8...	19	Ack
23	9.845587700	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	138	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
24	9.845598800	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	138	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
25	9.845645900			IEEE 8...	19	Ack
26	9.845654700			IEEE 8...	19	Ack
27	9.857613000	fe80::8123:343d:7e0...	fe80::14e2:ff5:d6a3...	DTLSv1...	83	Client Hello
28	9.857627800	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	83	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
29	9.857727900			IEEE 8...	19	Ack
30	9.857736800			IEEE 8...	19	Ack
31	9.875698500	fe80::14e2:ff5:d6a3...	fe80::8123:343d:7e0...	DTLSv1...	83	Hello Verify Request
32	9.875713900	fe80::14e2:ff5:d6a3...	fe80::8123:343d:7e0...	DTLSv1...	83	Hello Verify Request
33	9.875776400			IEEE 8...	19	Ack
34	9.875785900			IEEE 8...	19	Ack
35	9.885908700			IEEE 8...	19	Ack
36	9.885923500			IEEE 8...	19	Ack
37	9.900595600			IEEE 8...	19	Ack
38	9.900610000			IEEE 8...	19	Ack
39	9.900610000			IEEE 8...	19	Ack
26	9.845654700			IEEE 8...	19	Ack
27	9.857613000	fe80::8123:343d:7e0...	fe80::14e2:ff5:d6a3...	DTLSv1...	83	Client Hello
28	9.857627800	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	83	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
29	9.857727900			IEEE 8...	19	Ack
30	9.857736800			IEEE 8...	19	Ack
31	9.875698500	fe80::14e2:ff5:d6a3...	fe80::8123:343d:7e0...	DTLSv1...	83	Hello Verify Request
32	9.875713900	fe80::14e2:ff5:d6a3...	fe80::8123:343d:7e0...	DTLSv1...	83	Hello Verify Request
33	9.875776400			IEEE 8...	19	Ack
34	9.875785900			IEEE 8...	19	Ack
35	9.885908700			IEEE 8...	19	Ack
36	9.885923500			IEEE 8...	19	Ack
37	9.900595600			IEEE 8...	19	Ack
38	9.900610000			IEEE 8...	19	Ack
39	9.908260100	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	138	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
40	9.908272000	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	138	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
41	9.908315000			IEEE 8...	19	Ack
42	9.908324200			IEEE 8...	19	Ack
43	9.920739800			IEEE 8...	19	Ack
44	9.920769000			IEEE 8...	19	Ack
45	9.934259800	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	91	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
46	9.934279000	83:23:34:3d:7e:08:a...	16:e2:0f:f5:d6:a3:c...	6LoWPAN	91	Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
47	9.934322300			IEEE 8...	19	Ack
48	9.934328000			IEEE 8...	19	Ack
49	14.286186100	16:e2:0f:f5:d6:a3:c...	83:23:34:3d:7e:08:a...	6LoWPAN	139	Data, Src: 16:e2:0f:f5:d6:a3:cd:b0, Dst: 83:23:34:3d:7e:08:a3:90
50	14.286203500	16:e2:0f:f5:d6:a3:c...	83:23:34:3d:7e:08:a...	6LoWPAN	139	Data, Src: 16:e2:0f:f5:d6:a3:cd:b0, Dst: 83:23:34:3d:7e:08:a3:90
51	14.286243600			IEEE 8...	19	Ack
52	14.286254000			IEEE 8...	19	Ack
53	14.292520400			IEEE 8...	19	Ack
54	14.292528100			IEEE 8...	19	Ack
55	14.307279300	16:e2:0f:f5:d6:a3:c...	83:23:34:3d:7e:08:a...	6LoWPAN	138	Data, Src: 16:e2:0f:f5:d6:a3:cd:b0, Dst: 83:23:34:3d:7e:08:a3:90
56	14.307290500	16:e2:0f:f5:d6:a3:c...	83:23:34:3d:7e:08:a...	6LoWPAN	138	Data, Src: 16:e2:0f:f5:d6:a3:cd:b0, Dst: 83:23:34:3d:7e:08:a3:90
57	14.307335200			IEEE 8...	19	Ack
58	14.307351100			IEEE 8...	19	Ack

79	18.770689800		IEEE 8...	19 Ack
80	18.770695200		IEEE 8...	19 Ack
81	18.780431000	fe80::8123:343d:7e0... fe80::14e2:ff5:d6a3...	DTLSv1...	115 Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
82	18.780449200	83:23:34:3d:7e:08:a... 16:e2:0f:f5:d6:a3:c...	6LoWPAN	115 Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
83	18.780502500		IEEE 8...	19 Ack
84	18.780512700		IEEE 8...	19 Ack
85	20.555671000	fe80::14e2:ff5:d6a3... fe80::8123:343d:7e0...	DTLSv1...	114 Change Cipher Spec, Encrypted Handshake Message
86	20.555696400	fe80::14e2:ff5:d6a3... fe80::8123:343d:7e0...	DTLSv1...	114 Change Cipher Spec, Encrypted Handshake Message
87	20.555752000		IEEE 8...	19 Ack
88	20.555762300		IEEE 8...	19 Ack
89	20.570608000	83:23:34:3d:7e:08:a... 16:e2:0f:f5:d6:a3:c...	6LoWPAN	139 Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
90	20.570623100	83:23:34:3d:7e:08:a... 16:e2:0f:f5:d6:a3:c...	6LoWPAN	139 Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
91	20.570672400		IEEE 8...	19 Ack
92	20.570683600		IEEE 8...	19 Ack
93	20.581343600	fe80::8123:343d:7e0... fe80::14e2:ff5:d6a3...	DTLSv1...	69 Application Data
94	20.581360000	83:23:34:3d:7e:08:a... 16:e2:0f:f5:d6:a3:c...	6LoWPAN	69 Data, Src: 83:23:34:3d:7e:08:a3:90, Dst: 16:e2:0f:f5:d6:a3:cd:b0
95	20.581575100		IEEE 8...	19 Ack
96	20.581583300		IEEE 8...	19 Ack
97	20.595060300	fe80::14e2:ff5:d6a3... fe80::8123:343d:7e0...	DTLSv1...	88 Application Data
98	20.595076800	fe80::14e2:ff5:d6a3... fe80::8123:343d:7e0...	DTLSv1...	88 Application Data
99	20.595127000		IEEE 8...	19 Ack
100	20.595133900		IEEE 8...	19 Ack
101	20.605097500	fe80::8123:343d:7e0... fe80::14e2:ff5:d6a3...	DTLSv1...	78 Encrypted Alert
102	20.605146600	fe80::8123:343d:7e0... fe80::14e2:ff5:d6a3...	DTLSv1...	78 Encrypted Alert
103	20.605211100		IEEE 8...	19 Ack
104	20.605217100		IEEE 8...	19 Ack
105	20.614615000	fe80::14e2:ff5:d6a3... ff02::1	MLE	83
106	20.614631400	fe80::14e2:ff5:d6a3... ff02::1	MLE	83
107	20.626763700	fe80::14e2:ff5:d6a3... fe80::8123:343d:7e0...	DTLSv1...	78 Encrypted Alert
108	20.626780000	fe80::14e2:ff5:d6a3... fe80::8123:343d:7e0...	DTLSv1...	78 Encrypted Alert
109	20.626883800		IEEE 8...	19 Ack
110	20.626892200		IEEE 8...	19 Ack
111	20.644973900		IEEE 8...	19 Ack

Figure 4: Sniffer showing joining process.

Conclusions

Roberto Franco: Like the previous practice, this exercise helped me better understand the process of setting up a Thread-based network, including how messages are sent, devices are joined, and so on. We encountered several issues while capturing screenshots of the joining process; because Thread is a high-security protocol, the messages are encrypted, making it difficult to configure the application to read them.

Eduardo Guzman:

After doing this practice I feel like I can better understand how thread works and how I could possibly use then in future projects I may have this could also help me connect my microcontroller to other thread devices and maybe even connect it to the internet by using a border router.

Santiago Sancedo: On This practice I learned better the communication on thread, on how we used commands to connects the routers and how they communicate with the ip address I understand that better now.

References

1. NXP Semiconductors. (s.f.). *Thread: Introduction, network architecture, hands-on [Slide presentation]*. Course material provided by Estephania Martínez García.
2. NXP Semiconductors. (s.f.). *Thread: IPv6 addressing, management, routing, security, commissioning [Slide presentation]*. Course material provided by Estephania Martínez García.
3. Anthropic. (2026). *Claude Sonnet 5.5 [Large language model]*. <https://claude.ai>